

Rollout実機不具合 修正報告書

対象: DiffusionPolicy を用いた RealFairino5Demo (FR5実機) でのポリシー実行 (`bin/Rollout.py`) まわり

期間中に発見・修正した不具合を、発見の経緯・原因・対処の順にまとめる

目次

1. 全体概要
2. ① 状態(state)入力バグ — 方策に絶対座標が渡っていた
3. ② IK内部状態の巻き込み(windup)
4. ③ 速度クランプの経過時間が固定値だった
5. ④ ループのペーシングが正のフィードバックで暴走
6. ⑤ カメラのブロッキング待ちで制御ループが10Hzに律速
7. ⑥ 速度クランプの上限緩和が招いた起動時の急加速
8. ⑦ 速度クランプでの減速が閉ループを破綻させる (振動)
9. ⑧ タスク完了後に無限に同じ動作を繰り返す
10. ⑨ 絶対上限値が厳しすぎてタスクが完了不能に
11. ⑩ 起動直後の急加速 (バン!) — 観測の同期漏れ
12. ⑪ グリッパ状態読み取り警告のスパム
13. 検証したが効果がなかった対策
14. 新規に作成した診断ツール
15. 最終的な推奨設定

全体概要

DiffusionPolicyで学習したチェックポイントを実機のFR5アームで動かす過程で、次々と不具合が発覚した。症状は「推論ができていないように見える」「動きがガクガクする」「起動時に急加速する」「タスクを何度も繰り返す」など多岐にわたったが、調査の結果、原因は**モデルの学習内容そのものではなく、実機を動かすための周辺コード（状態の受け渡し・安全機構・タイミング制御）に潜む複数の独立した不具合**であったことが判明した。

本書では、発見された不具合を時系列に沿って解説する。多くの不具合は連鎖しており、ひとつを直すと別の不具合が表面化するという展開になったため、「なぜその修正が必要だったか」「その修正が次にどんな問題を引き起こしたか」を合わせて記載する。

#	不具合	深刻度	影響
①	状態入力に絶対座標を使用	重大	方策が学習時と全く異なる入力を受け取り、出力が支離滅裂に
②	IK内部状態のウィンドアップ	重大	速度制限下で指令が実機から際限なく乖離
③	速度クランプの経過時間が固定	高	推論の遅延ティックで指令が飢餓状態に
④	ペーシングの正のフィードバック	高	制御ループが2Hzまで低下
⑤	カメラ待ちで10Hz律速	高	1指令あたりの移動量が過大に
⑥	クランプ上限緩和で急加速	重大	実機で最大約10度の単発ジャンプ
⑦	速度クランプでの減速が振動を誘発	重大	クランプを絞るほど閉ループが破綻し激しく振動
⑧	タスク完了後に無限リピート	中	実機が報酬0のため自動停止せず永久に反復
⑨	絶対上限値0.5度でタスク完了不能	高	安全策のつもりが方策を分布外に押し出し停滞
⑩	起動直後の急加速（バン！）	重大	MoveJ直後に古い観測を使い14度の単発指令
⑪	グリッパ警告のスパム	軽微	起動前のコンソールが警告で埋まる（実害なし）

① 状態(state)入力バグ — 方策に絶対座標が渡っていた

重大

policy/diffusion_policy/RolloutDiffusionPolicy.py

他6ファイル

症状

学習時と同じカメラ画像を入力しても、実機での動きは「対象物の位置に関わらず同じ前後動作・グリッパ開閉を反復する」だけで、コップに近づく動作にすら至らなかった。オフライン検証（記録済みデータでの再現テスト）ではモデルは正しく軌道を再現できていたため、モデル自体は正常と判断していた。

原因

学習データはUMI（ハンドヘルド収録リグ）由来で、**ロールアウト開始時の手先姿勢を原点とする「エピソード相対座標」**で記録されている。実機側では

`RolloutBase.get_measured_data_for_policy()` がこの相対変換（絶対座標 → 相対座標）を正しく行うようになっていたが、`DiffusionPolicy`をはじめとする7つの方策クラスの独自の状態バッファ構築処理（`update_state_buf()` など）が、この変換メソッドを経由せず

`self.motion_manager.get_data(state_key, self.obs)` を直接呼び出しており、変換前の**絶対座標**をそのままモデルに渡していた。

```
# 修正前（バグ）：絶対座標をそのまま使用
convert_data_to_policy(
    self.motion_manager.get_data(state_key, self.obs), state_key
)

# 修正後：エピソード相対変換を経由
convert_data_to_policy(
    self.get_measured_data_for_policy(state_key), state_key
)
```

`dry_run`で実測した数値がこの深刻さを裏付けている。

修正前（絶対座標）の並進

(0.012, -0.267, 0.355) m

修正後（相対座標）の並進

(0, 0, 0) m

回転についても、修正前はクォータニオン (0.010, -0.003, -0.702, 0.713) と**90度以上傾いた値**が渡っていたのに対し、学習データの状態統計は回転がほぼ単位クォータニオン付近（`qw≈0.88~1.0`）でしか変化しない。つまりモデルは学習時に一度も見たことのない、分布から大きく外れた入力を毎回与えられていたことになる。

修正

影響を受けていた `RolloutDiffusionPolicy` / `RolloutDiffusionPolicy3d` /

`RolloutFlowPolicy` / `RolloutManiFlowPolicy` / `RolloutMlp` / `RolloutPi0` / `RolloutGr00t`

の7クラス全てで、状態読み取りを `get_measured_data_for_policy()` 経由に統一した。なお `RolloutAct` / `RolloutMtAct` / `RolloutSarnn` は独自の状態バッファを持たず、元々正しい経路 (`RolloutBase.get_state()`) を使っていたため影響を受けていなかった。

② IK内部状態の巻き込み(ワインドアップ)

重大

`common/body/ArmManager.py`

`common/manager/MotionManager.py`

`common/base/RolloutBase.py`

症状

速度制限 (`joint_vel_limit_scale`) を緩めるほど、揺れが**悪化**するという逆説的な現象が発生した。

原因

逆運動学 (IK) を担う `ArmManager.inverse_kinematics()` は、毎ティック自分自身の内部状態 `self.arm_joint_pos` を起点に1歩ずつ目標へ近づく計算を行い、その結果でまた `self.arm_joint_pos` を更新する。しかし**この内部状態は実機の実測関節角と一度も同期されていなかった**。IKは「指令通り完璧に動けた」前提で計算を進めるが、実際には安全機構の速度クランプで動きが制限されるため、内部モデルと実機の差は際限なく拡大していく (積分のワインドアップ)。

差が拡大すると、方策は「実機が大きく遅れた状態」を観測して**全く別の軌道を再計画**し、目標が逆方向へ飛ぶ。速度制限を緩めると、実機がこの「暴走した目標」により強く反応できてしまうため、症状が悪化していた。

修正

`ArmManager.sync_to_measured()` を新設し、実測関節角へ内部状態を強制的に合わせる仕組みを追加。 `MotionManager.sync_arm_to_measured(obs)` が全アームに対してこれを呼び出し、 `RolloutBase.run()` のメインループが毎ティック・指令計算の直前にこれを実行するようにした。以後、IKは常に「実機が本当にいる場所」を起点に計算される。

③ 速度クランプの経過時間が固定値だった

高

envs/real/RealEnvBase.py

症状

②の修正後もなお、関節角指令が跳ねる現象（1ティックで最大43度）が残っていた。

原因

安全機構（速度クランプ、`overwrite_command_for_safety`）は「今回の指令でどれだけ動いてよい」を `速度上限 × duration` で決めるが、この `duration` には常に固定値 `self.dt = 0.02秒` が渡されていた。実際の制御周期は拡散モデルの推論（実測最大281ms）やカメラ読み出し・XML-RPC通信の影響で26ms～472msと大きくばらついており、実時間で0.47秒経過したティックでも「0.02秒分（=1/23）しか動いてはいけない」と制限されていた。その結果、方策の目標に対する遅れが蓄積し、次に短い周期のティックが来た瞬間に一気に放出される「詰まる→跳ねる」を繰り返していた。

修正

`RealEnvBase.step()` で実際の経過時間を実測し（`_clamp_duration`）、速度クランプの計算にはその実測値を使うよう変更。ServoJ自体の `cmdT`（送信間隔）は元から実測経過時間を使う設計だったため、安全クランプ側もそれに合わせた形になる。

④ ループのペーシングが正のフィードバックで暴走

高

envs/real/RealEnvBase.py

症状

③の修正直後、動きは滑らかになったが、**ループ全体が極端に遅くなった**（2.0Hz、実測の59%のティックでアームが静止）。

原因

③の修正で使った `duration` という同一の変数が、実は速度クランプの計算だけでなく、**ループのペーシング**（`wait=True` のときの `sleep()` 時間の目標値）にも流用されていた。実測した経過時間をそのまま次のペーシング目標として渡してしまうと、「前回の周期と同じだけ待つ」という正のフィードバックが生まれ、周期はどんどん伸びて上限の0.5秒に張り付いてしまう。

修正

2つの用途を分離。`_set_action` に渡す `duration`（ペーシング目標）は公称値 `dt=0.02` に戻し、速度クランプの計算だけが専用の `_clamp_duration`（実測値）を参照するようにした。

⑤ カメラのブロッキング待ちで制御ループが10Hzに律速

高

envs/real/RealEnvBase.py

症状

④の修正後もなお、多少ましになった程度でカクつきが残っていた。制御周期の中央値がぴったり**0.100秒**という、外部クロックに同期したかのような不自然な値になっていた。

原因

`get_pointcloud_camera_data()` が、毎ステップ**新しいカメラフレームが届くまでブロック**して待っていた。Orbbecカメラの実際のフレームレートは10FPS（0.1秒間隔）であり、制御ループ全体がこれに完全に律速されていた。指令レートが10Hzしかないため、1回の指令でカバーしなければならない移動量が学習時（50Hz想定）の約5倍に膨らみ、これが飛びとして現れていた。

修正

カメラ待ちをノンブロッキング化。キューを最新フレームまでドレインしてから使用し、新フレームが無ければ直近のフレームを再利用して即座に返すようにした。実際にカメラが無応答になった場合（`RECONNECT_TIMEOUT_SEC = 2秒超`）のみ、従来通りフォールト扱いで自動リコネクトする。これにより制御ループはカメラのFPSから解放され、`dry_run`環境で約37~48Hzまで高速化した。

⑥ 速度クランプの上限緩和が招いた起動時の急加速

重大

envs/real/RealEnvBase.py

症状

⑤でループが高速化した直後の実機テストで、「バン！」という急加速が発生した。

原因

③で導入した速度クランプの上限 `STEP_DURATION_MAX_SEC = 0.5秒` が緩すぎた。ループが10Hzだった頃は目立たなかったが、⑤の修正で37Hzまで高速化したことで、「推論スパイクで0.31秒かかったティックの直後に、0.5秒分の移動量（速度30度/秒として約9.4度）を一度に許可してしまう」という危険な組み合わせが発生した。実測ログでも「実測との乖離が9.77度」というスパイクが確認された。

修正

`STEP_DURATION_MAX_SEC` を0.5秒から**0.1秒**に短縮し、単発の許可量を最悪でも3度程度に抑えた。あわせて、指令と実測の乖離が**30度を超えたら送信を停止して例外を送出するハードアボート**を新設し、何らかの理由で乖離が拡大した場合にアームがその場で保持されるようにした（従来は15度で警告を出すのみで、動作は継続していた）。

⑦ 速度クランプでの減速が閉ループを破綻させる（振動）

重大

envs/real/RealEnvBase.py

envs/operation/OperationRealFairino5Demo.py

症状

「動きを遅くしたい」という要望に対し `joint_vel_limit_scale`（速度クランプの倍率）を下げて対応したところ、**高速でガクガク振動する**という、意図と正反対の結果になった。

原因

速度クランプは「安全網」であって「速度調整の手段」ではなかった。実測波形が動かぬ証拠となった。

方策の目標（手首関節）： 63.7度 → 55.6度 → 48.1度 → 43.5度（急降下）
実測位置： 62.8度 → 63.0度 → 62.5度 → 57.3度（追従できず20度乖離）
次の目標： 53.2度（+9.7度、逆方向へジャンプ）

方策は最大75度/秒での動きを要求していたが、クランプは15度/秒しか許可しない。結果、①アームが目標から大きく遅れる → ②方策が「遅れた状態」を観測して**別の軌道を再計画** → ③目標が逆方向に飛ぶ、という閉ループの破綻が起きていた。クランプを絞れば絞るほど乖離が拡大し、振動が悪化する。

修正

`time_scale` パラメータを新設。制御周期そのものを引き伸ばし、**観測間隔とアーム速度を同じ比率で一緒に遅くする**方式に変更した。これにより方策が2つの観測間で条件付けている「空間的な変化量」が学習時と一致したまま保たれ、閉ループを壊さずに減速できる。これは実機で動作確認済みの `ReplayUmiOnFairino5.py --time_scale` と同じ考え方である。速度クランプ（`joint_vel_limit_scale`）は本来の役割である安全網に戻し、余裕を持たせた値（2.0）とした。

⑧ タスク完了後に無限に同じ動作を繰り返す

中

common/base/RolloutBase.py

症状

タスクが完了しても方策が停止せず、**何度も同じ動作を繰り返してしまう**。

原因

学習データは「開始姿勢を離れる→作業→開始姿勢へ戻る」という**往復構造**で記録されている。エピソード相対座標では、開始状態と終了状態がどちらも原点（単位姿勢）となり**数学的に区別がつかない**。模倣学習の方策自体には「完了」という概念がなく、実機では報酬(reward)が常に0のため既存の `auto_exit` 判定も機能しない。結果として、方策は「完了状態」を「新しいエピソードの開始」と誤認し、永久にタスクを繰り返す。

修正

`check_task_finished()` を新設。学習データ自身の統計（状態の並進レンジ）からしきい値を算出し、「**まず一定距離以上離れたことを確認してから、一定距離以内に戻ったことを検出する**」方式で往復完了を判定する。先に離脱を確認することで、「未開始」と「完了」の曖昧さを解消している。 -- `no_auto_finish` で無効化も可能。

⑨ 絶対上限値0.5度でタスクが完了不能に

高

envs/real/fairino5/RealFairino5EnvBase.py

症状

「一気に大きく動くと物が壊れる」という要望を受け、ServoJ送信直前に**1指令あたりの絶対移動量上限**（`max_joint_pos_delta_deg`）を新設し、安全側の値として0.5度を設定したところ、⑧の完了検出が働かず、タスクが**永久に終わらなくなった**。

原因

上限を厳しくしすぎると、指令が常に大きく遅れた状態になり、方策は「学習時には存在しなかった、目標から大きく取り残された状態」を観測し続けることになる。そのような状態からの正しい行動を学習していないため、**アームはその場で停滞**してしまう。実測では、目標地点（変位0.275m）まで到達した後、1499ステップ経過してもなお0.268m地点に留まり続けた（開始位置に戻るための帰還しきい値は0.057m）。安全のための制限が、方策を学習分布の外へ押し出していた。

修正

方策の実際の要求速度を計測（`time_scale=2.0`、クランプ無効時）した上で数値を再検討。

1ティックあたりの要求角度：95%点=0.20度 / 99%点=3.43度 / 最大=13.68度（稀なスパイク）

最大値を **2.0度** に設定。通常動作（0.2度）の10倍の余裕を持たせつつ、危険な13.7度スパイクは約7指令に分散させる。この設定でdry_run検証を行い、学習時（85ステップ）とほぼ同等の**65ステップで完走**することを確認した。

⑩ 起動直後の急加速（バン！） — 観測の同期漏れ

重大

```
envs/operation/OperationRealFairino5Demo.py
```

```
envs/operation/OperationRealFairino3Demo.py
```

```
envs/operation/OperationRealFairinoDualDemo.py
```

症状

②の修正（IK内部状態の同期）を入れた後の実機テストで、**ロールアウト開始直後にアームが急加速する現象が発生した。**

原因

これは②の修正が新たに生んだ副作用であった。 `MoveToInitPhase.start()` 内の `move_to_init_pose()` は **ブロッキングのMoveJでアームを大きく動かすが**、これは `env.step()` ループの**外側**で実行される。そのため `self.obs`（観測値）はMoveJ**実行前**の姿勢を保持したままになる。②で追加した毎ティックの同期処理がこの古い `obs` を使ってしまい、IKの内部状態をMoveJ前の姿勢へ巻き戻してしまう結果、最初の指令が「MoveJ前の位置へ戻れ」という**14度も離れた目標**になっていた。これを速度クランプの上限いっぱい（60度/秒）で実行しようとしたため、静止状態からの急加速となった。

修正

`move_to_init_pose()` の直後に `self.op.obs = self.op.env.unwrapped._get_obs()` を追加し、観測値を強制的に更新するようにした。同一構造を持つ3つのoperationクラス（Fairino5／Fairino3／FairinoDual）すべてに同じ修正を適用した。検証の結果、起動時の乖離は **13.82度 → 0.000度** に解消された。

⑪ グリッパ状態読み取り警告のスパム

軽微

envs/real/fairino5/RealFairino5EnvBase.py

症状

ロールアウト起動後、`n` キーを押す前の待機中に `Failed to read gripper DO state: no gripper command sent yet.` という警告が延々と流れ続けていた。

原因

`env.step()` はプロセス開始直後から毎ティック呼ばれるが、グリッパへの最初の指令は `move_to_init_pose()` / `GraspPhase` (`n` キー押下後) まで送られない。この「まだ指令を送っていない」という正常な起動時の状態を例外として `raise` し、それを毎ティック `print` していたため、実害のない警告が大量に表示されていた。

修正

「未指令」の状態を通常の分岐として扱うように変更し、警告を出さずに最後の既知値（デフォルト 50%）を静かに使うようにした。実際の異常（グリッパ読み取り失敗など）を知らせる別の警告文には影響していない。

検証したが効果がなかった対策

「動きがガクガクする」という残存課題に対して、以下の2つの対策を実装・検証したが、**いずれも測定の結果、効果が確認できなかったため、参考情報として記録する。**

方策の目標間を線形補間する

方策の目標が `skip` ティックごとにしか更新されず、その間は目標が一定に保持されるため、「動いて止まる」の階段状の動きになっているのではという仮説のもと、目標間を滑らかに補間する処理 (`get_interpolated_policy_action()`) を実装した。

動作中の停止ティック

0.0%

段差比(peak/median) 補間前

3.2

段差比(peak/median) 補間後

3.1

実測の結果、そもそもアームは止まっておらず、段差比もほぼ変化しなかった。

`max_joint_pos_delta_deg` による分散が既に目標を複数指令に分けていたため、補間の余地がなかったと考えられる。遅延が増えるだけなので、デフォルトでは無効化し (`--action_interp` で有効化可能)。

EMA平滑化（`command_smoothing_alpha`）の強化

指令に対する低域通過フィルタを強めれば滑らかになるのではという仮説のもと、係数を下げて検証した。

alpha	タスク完了	段差ばらつき (std/mean)
0.30（既定）	○ 完了	0.77
0.12（強め）	✕ 完了せず	0.75（改善なし）

平滑化を強めても改善は見られず、むしろ⑨と同種の「遅延しすぎて方策が分布外の状態を観測し停滞する」問題を引き起こした。残るムラは**方策自身が出力する軌道に内在するスパイク**であり、指令の後段のフィルタでは除去できないという結論に至った。既定値0.3のまま据え置いている。

新規に作成した診断ツール

`misc/CheckDiffusionPolicyPrediction.py`

学習に使ったエピソードの記録済み画像・状態を、学習時と全く同じ前処理でモデルに入力し、予測アクションと記録アクションを比較するオフライン検証スクリプト。実機を一切動かさずにチェックポイント自体の妥当性を検証できる。 `--image_mode all` を指定すると、画像を「実画像／最初のフレームで凍結／真っ黒」の3パターンに差し替えて予測を比較でき、**モデルが本当に画像に反応しているか**（状態だけのショートカット学習に陥っていないか）を検証できる。

`misc/RolloutWithRecordedImages.py`

`bin/Rollout.py` と全く同じ実機制御ループ（状態は実機から読み、指令は実機のServoJへ送る＝閉ループ）を使いつつ、ハンドカメラの画像だけを学習エピソードの記録映像に差し替えて動かすスクリプト。「モデルは正常なのに実機の挙動がおかしい」場合に、原因が**カメラ側のパイプライン**にあるのか**実機の制御ループ**にあるのかを切り分けるために作成した。本書の②～⑩の発見は、主にこのツールによるdry_run検証で行われた。

最終的な推奨設定

`envs/configs/RealFairino5DemoEnv_UmiPolicy.yaml` の最終的な設定値と、それぞれの役割をまとめる。

設定項目	値	役割
<code>time_scale</code>	2.0 (コマンドラインの <code>--time_scale</code> で上書き可)	速度調整はここで行う。 観測周期とアーム速度と一緒に伸ばすので閉ループが壊れない。実運用では8.0まで上げて滑らかさを確認済み。
<code>joint_vel_limit_scale</code>	2.0	安全網。速度調整には使わない。 絞ると閉ループが破綻し振動する (⑦参照)。余裕を持たせておき、 <code>hit_hard_clip</code> がほぼ0であることを確認する。
<code>max_joint_pos_delta_deg</code>	2.0	1指令あたりの絶対移動量上限。「一気に大きく動く」ことを防ぐ最終防衛線。厳しくしすぎるとタスクが完了不能になる (⑨参照) ため、下げる場合は必ず <code>dry_run</code> で完走確認する。
<code>command_smoothing_alpha</code>	0.3	EMA平滑化。既定値のまま。強めても効果がないことを確認済み。
追従誤差アポート	30度	指令と実測の乖離がこれを超えたら送信を停止するハードストップ (コード内定数)。
<code>--max_policy_steps</code>	既定: 学習エピソード最長×15	タスク完了検出 (⑧) が主。これは暴走時の保険。

本報告書は、実機に対する複数回の `dry_run` 検証および実機テストのログ解析に基づいて作成した。数値は特に断りのない限り、チェックポイント `RealUMIDemo_20260815_160937/policy_best.ckpt` を用いた検証結果である。